PSAI_112

Integrating Polkadot.js with Blackprint

- **Team Name:** Blackprint
- **Payment Address:** 0xE8b5923f891C5d42eBF9f385DDDFA4a8A74cb9AA (DAI)
- **Level:** 1

Project Overview :page_facing_up:

Overview

> Note: “node” does refer to Blackprint’s node, not blockchain’s node

Blackprint is a visual programming tool that can help developers interact with libraries by simply connecting nodes in realtime without the need to code. Developers themselves will need to use Blackprint Engine and load required modules to execute exported JSON of node, cable connections, and embedded data.

Blackprint hopefully can help the development of Metaverse, frontend development, as well as automation such as bots that connect to multiple blockchains. Blackprint itself is planning to expand to support several programming languages such as JavaScript, PHP, Golang, and Python. But this proposal is currently only for JavaScript as Blackprint is still trying to improve.

Project Details

Technology stack

- JavaScript (Browser & Node.js)
- Polkadot.js’s library
- Blackprint does have 2 main component:
  - Engine: Designed for managing data flow between nodes and can be run on Browser/Node.js/Deno
  - Sketch: Can be used for creating node editor and modify node connections
- **Blackprint Editor**: An IDE that combine the sketch and the engine and provide another tools on the GUI to make the development more easier

Architecture

Below is the current architecture on how Blackprint works.

> Note: `engine` does refer to Blackprint Engine, and not JavaScript’s V8 engine

Blackprint Sketch is a minimal node editor where the user can interact (move, click, manage cable connection) with the nodes and it’s depends on **Blackprint Engine**. Modifying node/cable from the sketch in real time will change engine’s behavior to manage data flow between nodes.

The data changes from the engine will trigger **JS Module** (the orange colored shape) for interact with Polkadot.js’s library. **JS Module** is an additional module/addons for Blackprint, when it’s loaded it will registering node for sketch and engine. Polkadot.js’s library then will handle the connection between parachain and the current environment (Browser/Node.js).

Minimum Viable Product (MVP)

> Please scroll down again if you prefer to see some video example instead

MVP

The screenshot above is taken from Blackprint Editor that using nodes from this MVP module. The module itself is already usable for browser, but the module haven’t been published on NPM. Because currently it’s still in development, I have put a few example to try it on the repository.

▶ Some explanation for the screenshot above

Example: Listening to new blocks

https://user-images.githubusercontent.com/11073373/148692354-3a9ffad4-ef5e-4fa5-9bdb-d8d07b5b59e9.mp4

With the current MVP, we can listen to new blocks/head by connecting to the parachain via WebSocket. By using Ctrl + Right Click on the **API** port, we can get a list of suggested node. The **Blocks Event** then will use the **API** to subscribe to `chain_subscribeNewHeads` with help of Polkadot.js’s library. The output value (**Number** port) of **Blocks Event** will be updated everytime the block number has changed, the **New** port is supposed for call a function/trigger. At the end of video, I also opened the Chain info explorer to see if the number was match with the current network state.

Example: Retrieve transaction fee/info

https://user-images.githubusercontent.com/11073373/148692778-bdf3e096-64d3-43e4-ac7c-38c7785de8cd.mp4

From the video, I’m creating a **Transfer** node that can be used for creating unsigned transaction. It will only executed after all required

port is filled. I'm then trying to retrieve the payment info for the **Txn** and sign it by using dummy signer. The **Txn Payment Info** node then will update it's value for the output port (Info). The output is an Object.

Limitation of the project:

- Blackprint doesn't generate JS script/code dynamically and can only operate if the custom node is already registered with Blackprint Engine from an external/loaded module.
- **Blackprint Sketch** can export the sketch into JSON, and the users can import it to **Blackprint Engine** for Browser/Node.js. The engine itself will need the **JS module** to interact with Polkadot.js's library.

Ecosystem Fit

Blackprint can be used as a playground for experimenting with Polkadot.js library. Like creating their own program without writing code for doing transaction, signing/encrypting data. But just by simply load the module and begin connecting nodes interactively. For production use, they can easily export to JSON and import it on their target environment (browser/Node.js).

Hopefully users with no knowledge about how create transaction into blockchain may find this helpful for getting started by learning the concept first. It can also help dApp and metaverse developers.

In the future of metaverse, I hope it can also help users who don't have programming background can easily program it's own logic with the available nodes realtime inside the game/world.

For example like non-programmer who want to: 1. Collaboratively creating in-game ATM that can connect to different blockchain via RPC 2. Trigger transaction when interacting with in-game shop and able to see how the data flow works with Blackprint

The above is just my vision for the future. I haven't created any prototype of it yet, but it may be possible if some Metaverse project adopted Blackprint for its project.

Team :busts_in_silhouette:

Team members

- StefansArya: Fullstack Engineer & Game Developer

Contact

- **Contact Name:** Stefanus Arya Cipta Gusnadika
- **Contact Email:** stefansarya1@gmail.com

Team's experience

I experienced to work as a freelancer since 2015. My experience with web development is about ±7 years since that. I'm also familiar with Golang, PHP, and Python. I usually create open source library on GitHub, and some closed project on GitLab.

Team Code Repos

- https://github.com/Blackprint
- https://github.com/ScarletsFiction
- https://github.com/StefansArya

Team LinkedIn Profiles

- https://www.linkedin.com/in/stefansarya

Development Status :open_book:

> Note: `Polkadot.js nodes for Blackprint` is the `JS module` I mentioned on Architecture section

At the time of writing this proposal. Currently Blackprint is already usable, but it's still unstable and being optimized for better performance and design. It's already trying to follow semantic versioning and may only introduce breaking changes on increment of v0.*.0.

Below is the repository about current development.

- Blackprint Engine for JavaScript
- Blackprint Sketch for Browser
- Blackprint Editor
- Polkadot.js nodes for Blackprint (the MVP itself)

Currently the MVP can be run on browsers. The support for Node.js is still work in progress and may be delivered on **Milestone 3**. The current MVP may already contain nodes that will be delivered for Milestone 1 and Milestone 2, I'm still preparing to create the unit test and some example before I deliver it.

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** ±6 months
- **Full-time equivalent (FTE):** 1 FTE
- **Total Costs:** 9,200 DAI

Milestone 1 — Connection and data encryption nodes for Browser

- **Estimated duration:** ±3 weeks
- **FTE:** 1
- **Costs:** 4,000 DAI

Number	Deliverable	Specification
0a.	License	MIT
0b.	Documentation	I will provide **inline documentation** of the code and simple example that can be imported to the Blackprint Editor
0c.	Testing Guide	Delivered node will be fully covered by unit tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	We will use GitHub Action/Workflow instead, for manual UI testing/interaction we can use Blackprint Editor
1.	HTTP & WebSocket node	Able to connect to parachain's test/mainnet (including: Polkadot/Kusama/Westend) by specifying the RPC URL
2.	Event node	Able to listen for new blocks/heads of the connected parachain node
3.	Mnemonic/seed importer node	Able to convert mnemonic into private key (seed) that can be used for decrypting data or signing data, the private key is in the Keyring
4.	Encrypt, Decrypt node	Able to encrypt data with public key, and able to decrypt data with private key. Public key and private key is in the Keyring
5.	Sign, Verify node	Able to sign data with private key, and able to verify data with public key. Public key and private key is in the Keyring
6.	Package	I will submit the JavaScript package/module to NPM registry so it can be loaded browser via CDN

Milestone 2 — Basic transaction nodes for Browser

- **Estimated Duration:** ±3 weeks
- **FTE:** 1
- **Costs:** 2,000 DAI

Number	Deliverable	Specification
0a.	License	MIT
0b.	Documentation	I will provide **inline documentation** of the code and simple example that can be imported to the Blackprint Editor
0c.	Testing Guide	Delivered node will be fully covered by unit tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	We will use GitHub Action/Workflow instead, for manual UI testing/interaction we can use Blackprint Editor
1.	Event node	Able to listen for balance changes for an account from WebSocket RPC that connected to parachain
2.	Browser Extension	Able to use browser extension for signing data and obtain account list
3.	Balance transfer node	Able to transfer balance from an account to another account where the unsigned transaction itself can be signed with Keyring's keypair or browser extension
4.	Payment info node	Able to obtain payment information from an unsigned transaction
5.	Package	I will submit the JavaScript package/module to NPM registry so it can be loaded browser via CDN

Milestone 3 — Add support for Node.js

- **Estimated Duration:** ±4 month
- **FTE:** 0.8
- **Costs:** 3,200 DAI

Number	Deliverable	Specification
0a.	License	MIT
0b.	Documentation	I will provide **inline documentation** of the code, add/improve nodes documentation from Milestone 1 and 2 for both Browser and Node.js
0c.	Testing Guide	Delivered node will be fully covered by unit tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	I will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone for Node.js
1.	Nodes	Delivered nodes from Milestone 1 and 2 can be run on Node.js environment, and exported JSON from the Blackprint Sketch

		can be imported and executed from Node.js
2.	Package	I will submit to NPM registry so it can imported and used for Node.js

Note: I may need time to also maintain Blackprint

Future Plans

- Add Blackprint node for some feature from Substrate Metadata
- Finishing Blackprint's roadmap
- Build more nodes for Blackprint to make it more suitable as a development tools or IDE
- Build a community server on Discord to grow from individual developer into a team
- Because Blackprint is also still unstable and polkadot.js library may also have some changes on the future, I'm willing to update and maintain the the delivered nodes.

At the time writting – when having many nodes and cable, users may find Blackprint looks complicated because currently they can't arrange the cable. Because of that, on the future I'm going to improve the UI/UX for managing cable.

Additional Information :heavy_plus_sign:

How did you hear about the Grants Program?
I heard about the Grants Program from Web3 Foundation's website when doing research about Polkadot's Parachain Rollout.

Additional information that I think may relevant to this application:

- I'm planning to implement custom node for ethers.js as I already know how to use the library, but currently I more motivated to Polkadot because of this Grants Program.
- When doing research about Arweave, I also created custom nodes for Arweave. It's already usable on Blackprint Editor and can be imported on Blackprint Editor together with other nodes. But currently it has no tests or documentation.